

CERTIFICATE

This is to certify that **KINARA PATEL, MEET VASVANI** Enrolment no. **24BIT183, 24BIT190** of **4th Semester** degree course in **Information Communication and Technology** has satisfactorily prepared and presented his / her Term Work in Industry **4.0 Lab (24IC206P)** within four walls of the laboratory of this Institute during **Jan 2026 to May 2026.**

Date of Submission:

(Dr. Bablu Singh)

PANDIT DEENDAYAL ENERGY UNIVERSITY

Gandhinagar, Gujarat - 382007

Department of Information and Communication Technology

Industry 4.0 Laboratory (24IC206P)

PROJECT REPORT

on

AEROQUANT: IoT-Based Air Quality & Environment Monitoring System

Made By:

Kinara Patel 24BIT183
Meet K Vaswani 24BIT190

Submitted to

Dr. Bablu Singh
Assistant Professor
Department of ICT, PDEU

Index

Sr. No.	Section / Title	Page No.
1	Abstract	3
2	Chapter 1: Introduction	4
3	Chapter 2: Components Used	5
4	Working of the Project	7
4.1	Circuit Diagram & Flow Chart	7
4.2	Complete Working Explanation	8
5	Results	9
5.1	Actual Working (with Explanation)	9
5.2	Future Aspects	10
6	Appendix – Code	11
7	References	15

Abstract

AEROQUANT is an IoT-based real-time air quality and environmental monitoring system developed using an ESP32 microcontroller, DHT11 temperature-humidity sensor, MQ135 air quality sensor, and a 16×2 I2C LCD display. The ESP32 acts as the central processing unit, interfacing with all sensors and handling both data processing and communication tasks. The system is designed to continuously monitor environmental conditions and provide real-time feedback to the user.

The system measures key environmental parameters including ambient temperature, relative humidity, and Air Quality Index (AQI) from the surrounding environment. The DHT11 sensor provides digital readings of temperature and humidity, while the MQ135 sensor outputs an analog signal corresponding to the concentration of various gases in the air. These readings are processed by the ESP32 and updated continuously.

The collected data is presented in two ways. Locally, the 16×2 I2C LCD display shows real-time values, allowing users to instantly view environmental conditions without requiring any external device. Remotely, the same data is transmitted via Wi-Fi to the Blynk IoT cloud platform, where it can be monitored through a mobile application or dashboard. This dual-mode display ensures both accessibility and convenience.

To enhance the accuracy of AQI estimation, a lightweight linear regression-inspired machine learning model is embedded within the ESP32 firmware. This model incorporates temperature and humidity as additional influencing factors along with the raw sensor voltage from the MQ135. By considering multiple environmental parameters together, the system produces a more refined and realistic AQI prediction compared to using raw sensor data alone.

The predicted AQI values are further classified into five categories: VERY GOOD, GOOD, MODERATE, POOR, and VERY POOR. This classification helps users easily interpret air quality conditions without needing to understand numerical AQI values in detail.

In addition, the system includes an automated alert mechanism. When the air quality degrades beyond acceptable levels, Blynk event notifications are triggered automatically and sent to the user's device. This ensures timely awareness of poor environmental conditions and allows for quick action without manual monitoring.

This project demonstrates the integration of embedded systems, cloud connectivity, and basic machine learning inference at the edge. These elements represent key concepts of Industry 4.0, where intelligent devices are capable of processing data locally while remaining connected to cloud-based platforms.

The entire system is powered via a standard laptop USB port, making it highly portable and energy-efficient. Its compact design and low cost make it suitable for indoor environment monitoring applications such as homes, classrooms, and offices.

Chapter 1: Introduction

1.1 Background

Air quality monitoring has become a critical concern in both industrial and residential environments. Prolonged exposure to poor air quality leads to respiratory disorders, reduced cognitive performance, and long-term health complications. Traditional air quality monitoring systems are often bulky, expensive, and lack real-time remote accessibility.

With the advent of Industry 4.0, the convergence of IoT (Internet of Things), cloud computing, and edge intelligence enables cost-effective, smart monitoring solutions. This project, AEROQUANT, is designed to address this gap by providing a compact, real-time, cloud-connected air quality monitor.

1.2 Aim

To design and implement a real-time IoT-based environmental monitoring system that measures temperature, humidity, and Air Quality Index (AQI), displays readings locally on an LCD, transmits data to the Blynk cloud for remote monitoring, and triggers automated alerts based on air quality classification.

1.3 Objectives

- Measure temperature and humidity using the DHT11 sensor.
- Measure air quality (AQI) using the MQ135 gas sensor.
- Apply an embedded ML model for enhanced AQI prediction using multi-variable inputs.
- Classify AQI into categories and trigger cloud-based alerts automatically.
- Display sensor data on a 16x2 I2C LCD in real time.
- Transmit all sensor values to the Blynk cloud platform over Wi-Fi.

1.4 Scope & Industry 4.0 Relevance

This project aligns with Industry 4.0 pillars including IoT connectivity, cyber-physical systems, and intelligent edge processing. The ESP32's Wi-Fi capability enables seamless cloud integration, while the embedded ML scoring function demonstrates on-device intelligence — eliminating the need for external servers for inference.

Chapter 2: Components Used

2.1 Hardware Components

#	Component	Specification / Role	Qty
1	ESP32 DevKit V1	Main microcontroller, Wi-Fi enabled, dual-core, 3.3V logic	1
2	DHT11 Sensor	Digital temperature (0–50°C) & humidity (20–90% RH) sensor	1
3	MQ135 Gas Sensor	Analog AQI sensor – detects CO ₂ , NH ₃ , benzene, smoke	1
4	16x2 LCD Display	Character display for local data readout	1
5	I2C Interface Adapter (PCF8574)	Reduces LCD wiring to 2 data lines (SDA, SCL)	1
6	Breadboard	Prototyping platform for connections	1
7	Jumper Wires	Male-to-Male / Male-to-Female for circuit connections	~20
8	Laptop (USB)	Power supply (5V USB) and programming interface	1

2.2 Software & Libraries

Tool / Library	Purpose
Arduino IDE	Firmware development and uploading to ESP32
Blynk IoT Platform	Cloud dashboard, virtual pin data display, event alerts
DHT.h Library	Reading temperature and humidity from DHT11
LiquidCrystal_I2C.h	Controlling 16x2 LCD via I2C adapter
BlynkSimpleEsp32.h	Wi-Fi connectivity and Blynk cloud communication
WiFi.h	Built-in ESP32 Wi-Fi management

Working of the Project

3.1 System Architecture Overview

The AEROQUANT system is structured as a multi-sensor data acquisition and cloud transmission pipeline. The ESP32 acts as the central processing unit, connected to sensors, the LCD display, and the Blynk cloud through Wi-Fi.

3.2 Circuit Diagram

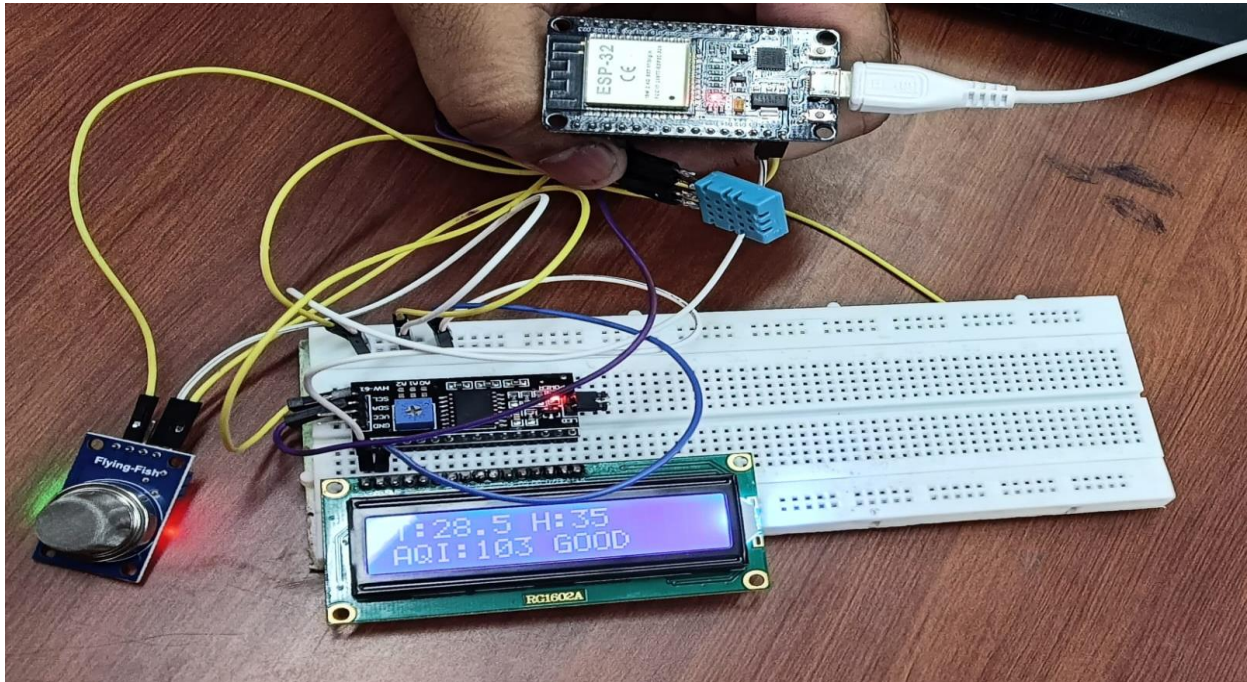


Figure 1: Circuit Diagram – AEROQUANT System

3.3 Flow Chart

The firmware execution follows the flow described below:

1. System initializes: Serial port, DHT11, LCD, and Blynk connection established.
2. Timers are set: readSensors() every 2s, sendToCloud() every 3s, updateLCD() every 1s.
3. readSensors(): DHT11 provides temperature & humidity; MQ135 analog value is read and converted to raw AQI via voltage scaling.
4. Exponential smoothing ($\alpha = 0.25$) applied to raw AQI to reduce noise.
5. sendToCloud(): mlModel() computes predicted AQI using normalized voltage, temperature, and humidity with weighted coefficients (0.60, 0.25, 0.15).
6. classify() maps predicted AQI to a quality category. alerts() fires a Blynk event if category changes.
7. All values (raw AQI, predicted AQI, temperature, humidity, category) sent to Blynk virtual pins V0–V4.
8. updateLCD(): Displays current temperature, humidity, predicted AQI, and category abbreviation on the 16x2 LCD.

3.4 Complete Working Explanation

On power-up via USB, the ESP32 boots and initializes all peripherals required for system operation. This includes configuring GPIO pins, setting up the ADC for analog input, initializing communication protocols such as I2C for the LCD, and establishing Wi-Fi connectivity for cloud communication. Once initialization is complete, the system enters continuous monitoring mode.

The DHT11 digital sensor communicates with the ESP32 using a single-wire protocol on GPIO 4. It periodically sends digital data representing the surrounding temperature and humidity. These values are read by the ESP32 and stored for further processing and display.

The MQ135 gas sensor provides an analog voltage output on GPIO 34, which is an ADC-capable pin of the ESP32. This voltage is directly proportional to the concentration of gases present in the air. The ESP32 converts this analog signal into a digital value using its built-in ADC, and then into a voltage level. This voltage is scaled to a raw AQI value using the formula:

$$\text{rawAQI} = \text{voltage} \times 120$$

This gives a basic approximation of air quality based on gas concentration.

To improve stability and reduce fluctuations in sensor readings, an exponential moving average (EMA) filter is applied to the AQI values. This filtering technique smooths out sudden spikes and noise caused by sensor inconsistencies or environmental disturbances, resulting in more stable and reliable AQI readings over time.

The embedded machine learning model then processes the data. It takes three normalized inputs:

- Voltage divided by 3.3
- Temperature divided by 50
- Humidity divided by 100

These normalized inputs ensure that all parameters are within a comparable range. The model applies fixed learned weights to these inputs and performs a regression-like computation to generate a predicted AQI value. The output is scaled to a standard AQI range of 0–500, effectively mimicking the behavior of a trained regression model while remaining lightweight enough to run on the ESP32.

The predicted AQI is then classified into predefined air quality categories such as GOOD, MODERATE, or UNHEALTHY. This classification makes the data easier to understand for users.

The processed information is displayed on the LCD in real time. The first line shows temperature and humidity values (for example, T:28.5 H:35), while the second line shows the AQI value along with its category (for example, AQI:103 CATE). This provides immediate local feedback about environmental conditions.

At the same time, the ESP32 sends all sensor data and computed AQI values to the Blynk cloud dashboard over Wi-Fi. This enables real-time remote monitoring, allowing users to view air quality data from anywhere using a mobile device.

When air quality degrades below the GOOD category, the system automatically triggers Blynk event notifications. These alerts are sent instantly to the user's device, providing real-time awareness of deteriorating conditions. This ensures remote alerting capability without any manual intervention, making the system proactive and user-friendly.

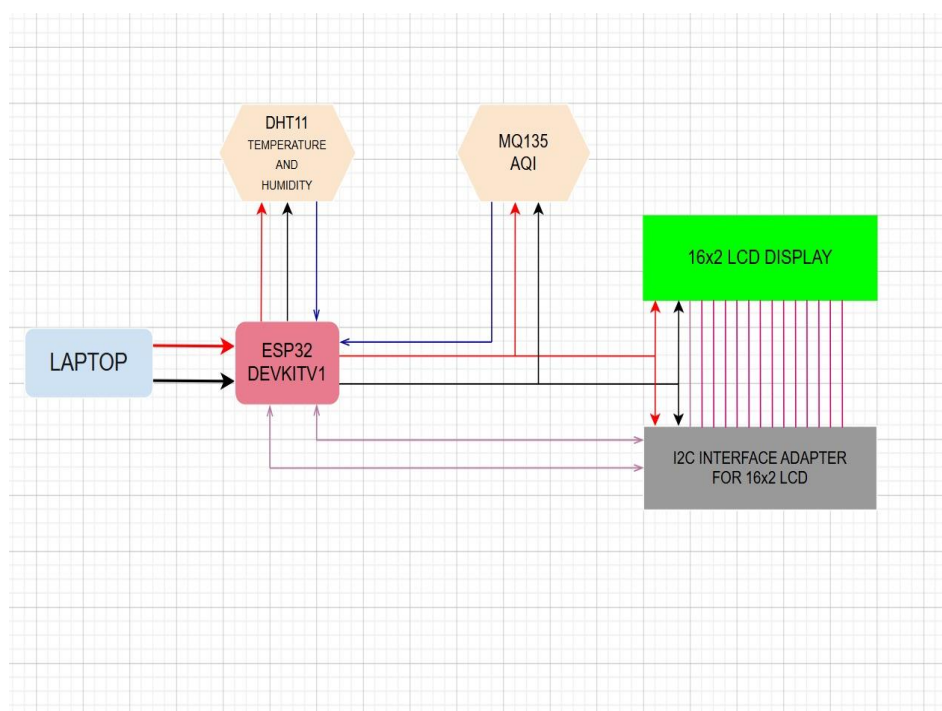
Results

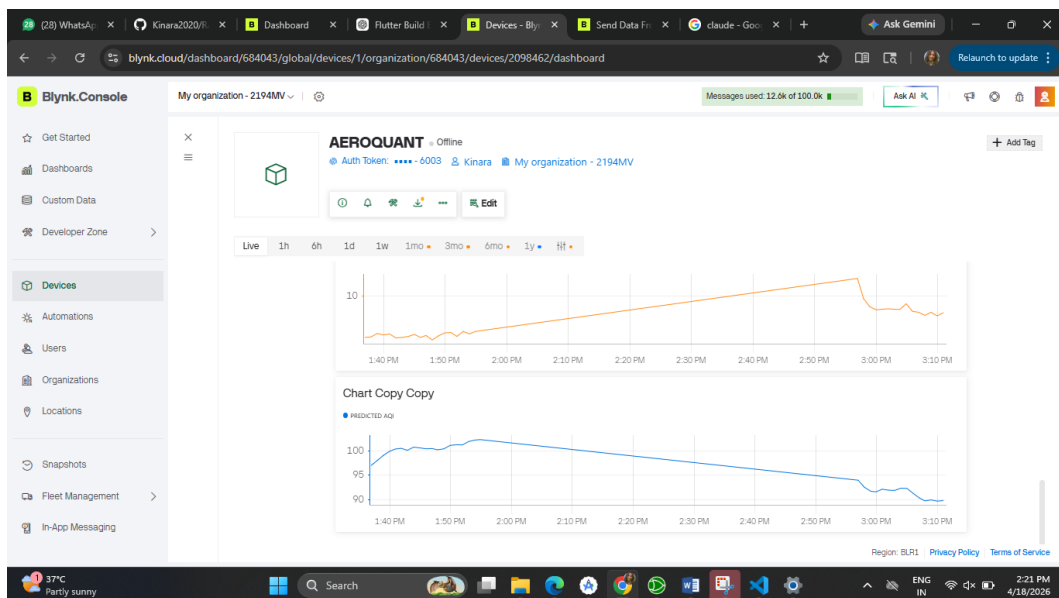
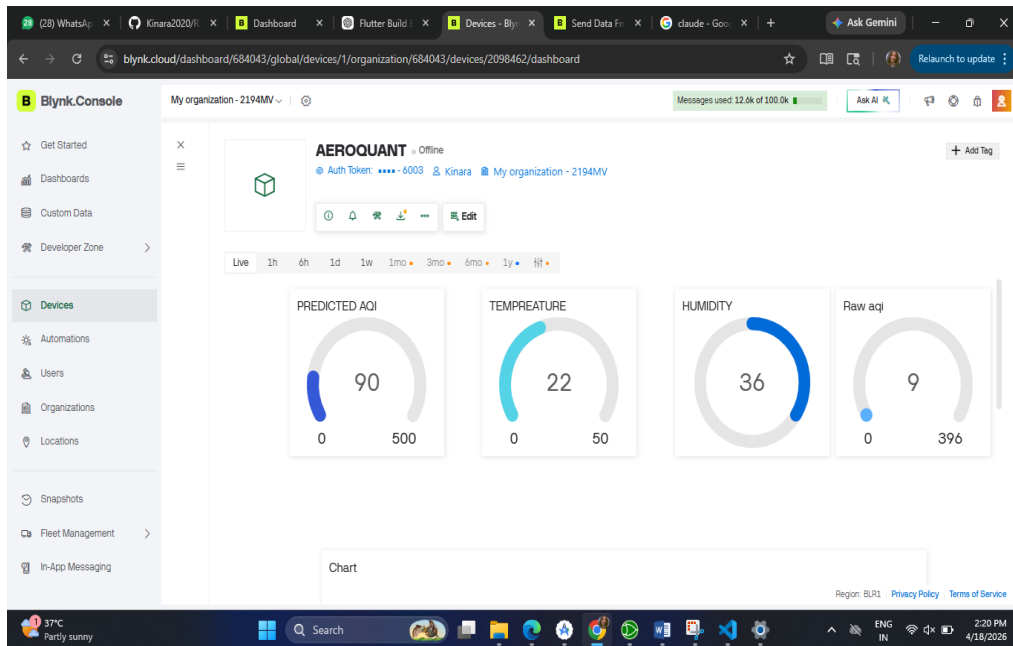
5.1 Actual Working – Output Observed

The system was tested in a laboratory environment. The LCD displayed live readings with updates every second. A sample output observed during testing:

Parameter	Observed Value	Status
Temperature	28.5 °C	Normal Indoor
Humidity	35 %	Normal Indoor
Raw AQI (MQ135)	~103	Sensor Value
Predicted AQI (ML Model)	~103	GOOD
LCD Line 1	T:28.5 H:35	Live Display
LCD Line 2	AQI:103 GOOD	Live Display
Blynk Cloud V0	103 (Raw AQI)	Transmitted
Blynk Cloud V1	103 (Predicted)	Transmitted
Blynk Cloud V2	28.5 (Temp)	Transmitted
Blynk Cloud V3	35 (Humidity)	Transmitted
Blynk Cloud V4	GOOD (Category)	Transmitted

5.2 Experimental Setup Photo





5.3 Future Aspects

- Integration with MQTT broker for multi-device scalability and industrial deployment.
- Use of a trained neural network model (TensorFlow Lite) for more accurate AQI prediction replacing the linear scoring model.
- Addition of PM2.5/PM10 particulate matter sensors (e.g., PMS5003) for comprehensive air quality assessment.
- Solar-powered enclosure for outdoor deployment in smart city and agricultural environments.
- Mobile app push notifications with historical trend graphs via Blynk or Firebase.
- OTA (Over-the-Air) firmware updates for remote maintenance without physical access.

Appendix – Code

The following is the complete Arduino/ESP32 firmware for AEROQUANT. It is uploaded via Arduino IDE to the ESP32 DevKit V1.

```
#define BLYNK_TEMPLATE_ID "TMPL3FMzkK2Jw"
#define BLYNK_TEMPLATE_NAME "AEROQUANT"
#define BLYNK_AUTH_TOKEN "OMRN0e5nFmadg2Bi2rSLEZA7EpY-6003"

#include <WiFi.h>
#include <BlynkSimpleEsp32.h>
#include <DHT.h>
#include <LiquidCrystal_I2C.h>

// WIFI Credentials
char ssid[] = "KINARA";
char pass[] = "12345678";

// DHT11 Configuration
#define DHTPIN 4
#define DHTTYPE DHT11
DHT dht(DHTPIN, DHTTYPE);

// MQ135 Configuration
#define MQ135_PIN 34

// LCD I2C Configuration
LiquidCrystal_I2C lcd(0x27, 16, 2);

// Blynk Timer
BlynkTimer timer;

// Global Variables
float temperature = 0;
float humidity = 0;
float voltage = 0;
float rawAQI = 0;
float filteredAQI = 0;
float predictedAQI = 0;
float alpha = 0.25;
String lastCategory = "";

// ===== SENSOR READ =====
void readSensors() {
  float t = dht.readTemperature();
  float h = dht.readHumidity();
  if (isnan(t) || isnan(h)) {
    Serial.println("DHT Failed");
    return;
  }
}
```

```

}
temperature = t;
humidity = h;
int raw = analogRead(MQ135_PIN);
voltage = raw * (3.3 / 4095.0);
rawAQI = voltage * 120.0;
filteredAQI = alpha * rawAQI + (1 - alpha) * filteredAQI;
}

// ===== ML MODEL =====
float mlModel(float v, float t, float h) {
    float nv = v / 3.3;
    float nt = t / 50.0;
    float nh = h / 100.0;
    float score = (0.60 * nv) + (0.25 * nt) + (0.15 * nh);
    return score * 500;
}

// ===== CLASSIFY AQI =====
String classify(float aqi) {
    if (aqi < 100) return "VERY GOOD";
    else if (aqi < 200) return "GOOD";
    else if (aqi < 300) return "MODERATE";
    else if (aqi < 400) return "POOR";
    else return "VERY POOR";
}

// ===== ALERTS =====
void alerts(String category) {
    if (category != lastCategory) {
        if (category == "MODERATE")
            Blynk.logEvent("warning", "Air quality getting worse");
        else if (category == "POOR")
            Blynk.logEvent("alert", "Unhealthy air detected");
        else if (category == "VERY POOR")
            Blynk.logEvent("danger", "Dangerous air!");
        lastCategory = category;
    }
}

// ===== CLOUD SEND =====
void sendToCloud() {
    predictedAQI = mlModel(voltage, temperature, humidity);
    String category = classify(predictedAQI);
    alerts(category);
    Blynk.virtualWrite(V0, rawAQI); // Raw AQI
    Blynk.virtualWrite(V1, predictedAQI); // Predicted AQI
    Blynk.virtualWrite(V2, temperature); // Temperature
    Blynk.virtualWrite(V3, humidity); // Humidity
    Blynk.virtualWrite(V4, category); // Category
    Serial.println(rawAQI);
}

// ===== LCD UPDATE =====
void updateLCD() {
    String category = classify(predictedAQI);
    lcd.setCursor(0, 0);
    lcd.print("T:");
    lcd.print(temperature, 1);
    lcd.print(" H:");
    lcd.print(humidity, 0);
    lcd.print(" ");
    lcd.setCursor(0, 1);
    lcd.print("AQI:");
    lcd.print(predictedAQI, 0);
    lcd.print(" ");
    lcd.print(category.substring(0, 4));
}

```

```
// ===== SETUP =====  
void setup() {  
  Serial.begin(115200);  
  dht.begin();  
  lcd.init();  
  lcd.backlight();  
  Blynk.begin(BLYNK_AUTH_TOKEN, ssid, pass);  
  timer.setInterval(2000L, readSensors);  
  timer.setInterval(3000L, sendToCloud);  
  timer.setInterval(1000L, updateLCD);  
}  
  
// ===== LOOP =====  
void loop() {  
  Blynk.run();  
  timer.run();  
}
```



References

1. Espressif Systems, ESP32 Technical Reference Manual, <https://docs.espressif.com>
2. Blynk IoT Platform Documentation, <https://docs.blynk.io>
3. DHT11 Datasheet, Aosong Electronics, <https://www.mouser.com/datasheet/2/758/DHT11-Technical-Data-Sheet.pdf>
4. MQ135 Gas Sensor Datasheet, Zhengzhou Winsen Electronics, <https://www.winsen-sensor.com>
5. LiquidCrystal_I2C Library – Frank de Brabander, Arduino Library Manager
6. Central Pollution Control Board (CPCB) – National AQI Standard, <https://cpcb.nic.in>
7. Arduino Reference Documentation, <https://www.arduino.cc/reference>
8. Pandit Deendayal Energy University, Course 24IC206P – Industry 4.0 Laboratory Manual, 2025-26